CodeVerse Soban

AI Agent Factory — Level 1 Exam

Loop Engineering — Complete MCQ Practice

100 MCQs · 4 Options Each · Correct Answer + Explanation Included

---

Q1. Boris Cherny, who built Claude Code, described a fundamental shift in how he works. What did he say replaced prompting as his primary activity?

A) Writing SKILL.md files for every project

B) 'I don't prompt Claude anymore. I have loops running that prompt Claude — my job is to write loops.'

C) Reviewing AI-generated pull requests manually each morning

D) Designing evaluation rubrics for AI output quality

✔ Correct Answer: B) 'I don't prompt Claude anymore. I have loops running that prompt Claude — my job is to write loops.'

Explanation: Cherny's quote is the course's founding premise: the important skill moved from writing prompts to designing loops that do the prompting for you.

Q2. What are the TWO things the course says a loop can never do for you, and where does your value go?

A) Debugging and deploying — your value moves to code review and testing

B) Intent (stating clearly what you want, clear enough to be checked) and accountability (standing behind the result) — the loop handles the steps in the middle

C) Scheduling and monitoring — your value moves to writing the initial prompt

D) Token management and context control — your value moves to model selection

✔ Correct Answer: B) Intent (stating clearly what you want, clear enough to be checked) and accountability (standing behind the result) — the loop handles the steps in the middle

Explanation: 'Intent and accountability stay yours. The loop handles the steps in the middle.' This is the course's central principle stated in Concept 1.

Q3. The course describes two loops that share one name. What is the critical difference between the small (inner) loop and the big (outer) loop?

A) The small loop runs inside Claude Code; the big loop runs inside OpenCode

B) The small loop stops when the MODEL decides it is finished; the big loop is the outer cycle that decides which task to give the worker, when to start, how to grade the result, and what to remember

C) The small loop handles code tasks; the big loop handles document tasks

D) The small loop uses subagents; the big loop uses a single agent

✔ Correct Answer: B) The small loop stops when the MODEL decides it is finished; the big loop is the outer cycle that decides which task to give the worker, when to start, how to grade the result, and what to remember

Explanation: The small loop stops on the model's own opinion (the break condition). The big loop is the manager — it wraps and orchestrates many small-loop runs. One full run of the small loop is just one beat of the big loop.

Q4. Why is the small loop's only stop condition — the model's own opinion — described as a problem?

A) Because it causes the model to use too many tokens on every run

B) Because the agent that performed the work should not approve its own result — a model may say 'Done! All fixed' and stop without ever running the tests

C) Because the model cannot understand complex stopping criteria written in plain English

D) Because the model's opinion changes with each run, making results unpredictable

✔ Correct Answer: B) Because the agent that performed the work should not approve its own result — a model may say 'Done! All fixed' and stop without ever running the tests

Explanation: The course gives this exact failure: the agent changes a file, writes 'Done! All fixed,' and stops — but never ran the tests. This is why outside stops exist: checked conditions, limits, no-progress checks, and a separate checker.

Q5. The course describes four engineering layers that evolved over time. Which is the CORRECT order from innermost to outermost?

A) Context engineering → Prompt engineering → Harness engineering → Loop engineering

B) Loop engineering → Harness engineering → Context engineering → Prompt engineering

C) Prompt engineering → Context engineering → Harness engineering → Loop engineering

D) Harness engineering → Loop engineering → Prompt engineering → Context engineering

✔ Correct Answer: C) Prompt engineering → Context engineering → Harness engineering → Loop engineering

Explanation: The four layers in order: prompt engineering (the words you send), context engineering (everything the model sees in one turn), harness engineering (the code around the model running tools), loop engineering (the outer cycle). Each became the industry's focus roughly a year apart.

Q6. The course says a loop has five working parts plus one saved-memory layer. What is the sixth part called, and what does it do?

A) The checkpoint — it saves the agent's model weights between runs

B) The spine — a file on disk (or a board like Linear) that records what is done and what comes next, so today's run knows what yesterday's run did

C) The manifest — a configuration file listing all connectors and subagents

D) The ledger — a token-cost tracker that monitors spending across beats

✔ Correct Answer: B) The spine — a file on disk (or a board like Linear) that records what is done and what comes next, so today's run knows what yesterday's run did

Explanation: The spine is the one part beginners skip. The model forgets everything between runs. The spine is how memory persists — without it, the loop just repeats its first step forever.

Q7. In the course's body metaphor, what does the term 'beat' refer to?

A) The time interval between scheduled runs

B) One complete run of the loop — one full pass through all its steps

C) The maker-checker verification step inside a single agent turn

D) The heartbeat pulse that starts the loop each morning

✔ Correct Answer: B) One complete run of the loop — one full pass through all its steps

Explanation: A beat = one complete firing of the loop. The heartbeat MAKES beats happen. Every other part of the loop describes what happens INSIDE one beat.

Q8. The course uses a kitchen analogy. What do connectors represent, and what do skills represent?

A) Connectors = the recipe cards; skills = the kitchen tools and stocked pantry

B) Connectors = the kitchen (tools and stocked pantry with real ingredients); skills = the recipe cards pinned to the wall

C) Connectors = the chef's instructions; skills = the ingredients purchased each day

D) Connectors = the customers' orders; skills = the kitchen staff who fulfill them

✔ Correct Answer: B) Connectors = the kitchen (tools and stocked pantry with real ingredients); skills = the recipe cards pinned to the wall

Explanation: From the earlier courses, carried into the loop: connectors are the kitchen (what AI can reach — Drive, Slack, GitHub), skills are the recipe cards (how AI should behave). The same analogy applies here.

Q9. What is the difference between a 'trigger' and a 'heartbeat' in loop engineering terminology?

A) A trigger is manual; a heartbeat is always automated

B) Trigger/fire means to start a run; heartbeat is the schedule, event, or condition that starts a beat — they describe the same mechanism from two angles

C) A trigger is a GitHub event; a heartbeat is a time-based schedule

D) A trigger starts a single agent; a heartbeat starts a multi-agent system

✔ Correct Answer: B) Trigger/fire means to start a run; heartbeat is the schedule, event, or condition that starts a beat — they describe the same mechanism from two angles

Explanation: The glossary defines both: 'Trigger/fire: to start a run' and 'Heartbeat: the schedule, event, or condition that starts a beat.' They describe the same starting mechanism.

Q10. Before July 2026, what was the limitation of running loops on the web (claude.ai, chatgpt.com)?

A) Web-based loops could only run for a maximum of 60 minutes

B) You could not run a loop in the chat box, because it waited for your turn every step — you were the schedule. Real unattended loops required leaving the browser for a tool that could fire on its own

C) Web interfaces did not support subagents or maker-checker patterns

D) Token limits on web interfaces were too low to sustain multi-beat loops

✔ Correct Answer: B) You could not run a loop in the chat box, because it waited for your turn every step — you were the schedule. Real unattended loops required leaving the browser for a tool that could fire on its own

Explanation: The course is precise: 'you cannot run a loop in the chat box, but the web pages around it can.' Before July 2026, a chat box waited for you every turn — you were the heartbeat. After July 2026, Claude Cowork at claude.ai changed this.

Q11. What is the defining characteristic of an in-session loop (Concept 4) that makes it unsuitable for running while you sleep?

A) It cannot access file systems or execute commands during its runs

B) The timer lives inside the open session — close the terminal and the watching dies with it, because nothing external keeps it alive

C) It has a maximum runtime of 30 minutes per beat

D) It requires manual approval before each beat can execute

✔ Correct Answer: B) The timer lives inside the open session — close the terminal and the watching dies with it, because nothing external keeps it alive

Explanation: In-session loop = timer inside the session. The ISS project illustrates this: close the terminal and the watching stops. This is 'the definition of an in-session loop, and it is the reason Concepts 6 and 7 exist at all.'

Q12. What is the critical difference between a fixed-timer loop and a conditional (run-until-done) loop?

A) A fixed-timer loop runs indefinitely; a conditional loop has a maximum of 10 iterations

B) A fixed-timer loop repeats at a chosen interval regardless of outcome; a conditional loop stops when a specific condition becomes true — it does not know when to stop by the clock, only by the result

C) Fixed-timer loops use schedules; conditional loops use event-driven triggers

D) Fixed-timer loops run on Claude Code; conditional loops run on OpenCode

✔ Correct Answer: B) A fixed-timer loop repeats at a chosen interval regardless of outcome; a conditional loop stops when a specific condition becomes true — it does not know when to stop by the clock, only by the result

Explanation: 'A fixed-timer loop does not know when the work is complete. A conditional loop stops because the work is complete.' The key: 'A separate command or checker must decide this.'

Q13. The /goal command in Claude Code uses a separate smaller model (Haiku by default) as a checker. What CANNOT this checker do, and why does the loop require visible evidence?

A) The checker cannot access the internet, so all evidence must be pre-cached

B) The checker cannot run commands — it can only read the conversation, so the worker must run tests and show results in its output; without visible evidence the checker cannot confirm the goal was met

C) The checker cannot process more than 4,000 tokens per check, so outputs must be summarized

D) The checker cannot evaluate code quality, only count lines changed

✔ Correct Answer: B) The checker cannot run commands — it can only read the conversation, so the worker must run tests and show results in its output; without visible evidence the checker cannot confirm the goal was met

Explanation: 'The checker cannot run commands. It can only read the conversation. The worker must therefore run the tests and show the results in its output. Without visible evidence, the checker cannot confirm that the goal was met.'

Q14. Every loop needs three stops. What is the 'no-progress check,' and what specific failure does it prevent?

A) A check that prevents the loop from running if no new commits were pushed since the last beat

B) A check that catches when the agent repeats the same action with the same arguments — it is stuck, and retrying won't fix it — so the whole token limit isn't wasted repeating one mistake

C) A check that verifies no team members are actively editing the files before the loop starts

D) A check that confirms the API rate limits have not been exceeded before the next beat

✔ **Correct Answer: B) A check that catches when the agent repeats the same action with the same arguments — it is stuck, and retrying won't fix it — so the whole token limit isn't wasted repeating one mistake**

Explanation: Three stops: success condition, limit, and no-progress check. The no-progress check catches the agent going in circles with identical actions — prevents burning the entire token budget on one repeated mistake.

Q15. What is a 'doom loop,' and what are the three defenses the course recommends?

A) A loop with no stopping condition that runs until the account is empty; defended by setting a spend cap

B) A messy context leading to worse decisions, which add more mess, which makes the next decision worse still; defended by compacting long runs, moving big outputs to files, and handing messy subtasks to a subagent

C) A loop that accidentally deletes files; defended by working on copies, dry runs, and scoped folders

D) A loop that fires too frequently, overloading the API; defended by rate limiting, exponential backoff, and circuit breakers

✔ **Correct Answer: B) A messy context leading to worse decisions, which add more mess, which makes the next decision worse still; defended by compacting long runs, moving big outputs to files, and handing messy subtasks to a subagent**

Explanation: 'A doom loop: a messy context leads to a worse decision, which adds more mess, which makes the next decision worse still.' Three defenses: compact long runs, move big outputs to files, hand messy subtasks to a subagent with its own context.

Q16. What is the difference between a 'Routine' and a 'scheduler' in Claude Code terminology?

A) A Routine runs locally; a scheduler runs in the cloud

B) A scheduler is the general concept (any always-on clock that launches a fresh run on time — cron, GitHub Actions, cloud service). A Routine is Claude Code's own cloud-hosted version, where Anthropic provides the clock and the machine

C) A Routine is limited to 5 runs per day; a scheduler has no limit

D) A Routine can use connectors; a scheduler cannot access external services

✔ **Correct Answer: B) A scheduler is the general concept (any always-on clock that launches a fresh run on time — cron, GitHub Actions, cloud service). A Routine is Claude Code's own cloud-hosted version, where Anthropic provides the clock and the machine**

Explanation: 'Every Routine is a scheduler. Not every scheduler is a Routine.' The Routine is Claude Code's specific cloud-hosted product. Cron and GitHub Actions are also schedulers but not Routines.

Q17. The course says 'one-off scheduled runs do not count against the daily Routine cap.' Why is this feature specifically useful for loop builders?

A) It allows unlimited testing without any cost implications

B) It lets you fire a one-off run immediately to prove the prompt behaves before committing it to a recurring schedule — 'prove it fast and watched before you trust it slow and unattended'

C) It bypasses all connector permissions for testing purposes

D) It grants access to frontier models for a single run regardless of plan

✔ **Correct Answer: B) It lets you fire a one-off run immediately to prove the prompt behaves before committing it to a recurring schedule — 'prove it fast and watched before you trust it slow and unattended'**

Explanation: The Sky Watch project illustrates this: '/schedule in 2 minutes, run the sky-watch skill' — a one-off to rehearse without waiting for midnight. One-offs let you prove behavior without spending your daily cap.

Q18. The course says a scheduled loop prompt should include 'for today' rather than 'for the week ahead.' Why does this matter?

A) Weekly forecasts require more tokens and would exceed context limits

B) A daily run with a weekly window re-sends yesterday's forecast every morning — matching the window to the cadence prevents the loop from reporting stale data on every fire

C) The AI cannot accurately predict information more than 24 hours in advance

D) Weekly ranges cause date parsing errors in most API responses

✔ **Correct Answer: B) A daily run with a weekly window re-sends yesterday's forecast every morning — matching the window to the cadence prevents the loop from reporting stale data on every fire**

Explanation: 'A daily run should report the day it fires, or it just re-sends yesterday's forecast every morning. Match the window to the cadence.' This is from the Sky Watch project in Concept 6.

Q19. What is the key reason a Channel (used for chat-message events in Claude Code) cannot work when your machine is closed?

A) Channels require constant internet connectivity that a closed laptop cannot provide

B) A Channel drops messages into a session that is already running — it needs a live session (an open terminal or background session), so unlike cloud Routines it doesn't work with the machine off

C) Channels are limited to 10 messages per hour and require manual acknowledgment

D) The Channel authentication token expires when the session closes

✔ Correct Answer: B) A Channel drops messages into a session that is already running — it needs a live session (an open terminal or background session), so unlike cloud Routines it doesn't work with the machine off

Explanation: 'It needs a live session. The session must already be running (an open terminal, or a background session), so a Channel does not work with the machine off.' This is its one weakness compared to Routines.

Q20. In Claude Code, what is the distinction between a PR-trigger Routine and the Claude Code GitHub Action for handling GitHub events?

A) Routines handle PRs; the GitHub Action handles issue comments

B) The Claude Code GitHub Action in CI does the same unattended job as a GitHub-trigger Routine and needs no research-preview access and no daily run cap — a Pro or Max plan is enough. It is the cheapest door into unattended work

C) The GitHub Action runs on your local machine; Routines run in the cloud

D) Routines support push events; the GitHub Action only supports PR events

✔ Correct Answer: B) The Claude Code GitHub Action in CI does the same unattended job as a GitHub-trigger Routine and needs no research-preview access and no daily run cap — a Pro or Max plan is enough. It is the cheapest door into unattended work

Explanation: 'anthropics/claude-code-action@v1 in a GitHub Actions workflow does the same job, and needs no research-preview access and no daily run cap... It is the cheapest door into unattended work.'

Q21. The course asks: 'does it end, or does it repeat?' What does each answer tell you about the correct heartbeat to use?

A) Ends → use a schedule; repeats → use a conditional loop

B) Ends and a command can prove the end → conditional loop. Repeats → schedule or event. Happens once → no loop at all (an ordinary session is the right tool)

C) Ends → use an in-session loop; repeats → use cloud Routines

D) Ends → use the API trigger; repeats → use the GitHub event trigger

✔ Correct Answer: B) Ends and a command can prove the end → conditional loop. Repeats → schedule or event. Happens once → no loop at all (an ordinary session is the right tool)

Explanation: This is the course's decision rule for choosing heartbeats: 'The task ends, and a command can prove the end → a conditional loop. The task repeats → a schedule or an event. The task happens once → no loop at all.'

Q22. The Doorbell project (Concept 7) demonstrates something about the need for tokens/credentials in unattended loops. What is the rule it illustrates?

A) All loops require OAuth authentication regardless of where they run

B) Your laptop already knew who you were; a rented stranger (GitHub runner, cloud session) does not — needing credentials and surviving a closed lid are the same fact seen from two sides

C) Token storage in secrets is more secure than environment variables for unattended loops

D) Credentials must be rotated after each run to prevent session hijacking

✔ Correct Answer: B) Your laptop already knew who you were; a rented stranger (GitHub runner, cloud session) does not — needing credentials and surviving a closed lid are the same fact seen from two sides

Explanation: 'That also explains the token. Your laptop already knew who you were. A rented stranger does not. Every unattended loop pays this price.' This is why GitHub Actions and Routines need setup-token or API keys.

Q23. Why do two separate pushes to the same pull request result in two completely independent sessions in event-driven Routines?

A) GitHub throttles simultaneous sessions on the same repository to prevent conflicts

B) Each matching event starts its own fresh session, so two pushes to one PR are two sessions that know nothing about each other — which is exactly why the spine must live in the repo, not the session

C) The second session automatically inherits context from the first session via session linking

D) Claude Code queues events and processes them sequentially in one continuous session

✔ Correct Answer: B) Each matching event starts its own fresh session, so two pushes to one PR are two sessions that know nothing about each other — which is exactly why the spine must live in the repo, not the session

Explanation: 'Note the pattern in every row but the Channel: each event starts a fresh session, so two events know nothing about each other.' This is why the spine is critical for event-driven loops.

Q24. What problem does a git worktree solve, and why does it matter specifically in loops?

A) It prevents token limits from being exceeded when multiple files are read simultaneously

B) It gives each parallel agent a separate working folder on its own branch, so two agents working at the same time cannot overwrite each other's files

C) It creates a backup copy of the repository before any automated changes are made

D) It limits each agent to a specific subdirectory of the repository for security

✔ Correct Answer: B) It gives each parallel agent a separate working folder on its own branch, so two agents working at the same time cannot overwrite each other's files

Explanation: 'The moment a loop runs more than one agent at once, they start to overwrite each other's files... A git worktree fixes this. It is a separate working folder, on its own branch, that shares the same repo history.'

Q25. The course says a skill keeps loop prompts tiny. What specific architecture pattern does it recommend?

A) Put all loop logic in the Routine's prompt and keep the skill file under 100 lines

B) Don't paste a long block of instructions into a schedule nobody will keep up to date — make the scheduled prompt one line ('run the daily-triage skill') and put the detail in the skill. Short loop prompt, logic that is easy to update, lower token cost on every beat

C) Split skills into one file per step and load them sequentially during the beat

D) Store skills in a cloud database that the loop fetches on each run to ensure they are always current

✔ Correct Answer: B) Don't paste a long block of instructions into a schedule nobody will keep up to date — make the scheduled prompt one line ('run the daily-triage skill') and put the detail in the skill. Short loop prompt, logic that is easy to update, lower token cost on every beat

Explanation: 'Do not paste a long block of instructions into a schedule that nobody will keep up to date. Instead, your scheduled prompt becomes one line... and the skill holds the detail. Short loop prompt, logic that is easy to update, lower token cost on every beat.'

Q26. The course gives three rules for connectors in a loop that differ from using connectors by hand. What are they?

A) Always prefer REST APIs over MCP, authenticate with API keys rather than OAuth, and log every connector call to a file

B) Fewer focused tools beat many overlapping ones (model loses track of which fits); writes must be safe to repeat (a retried create makes a duplicate); errors must say what to do next (the error message is the input to the next beat)

C) Connectors must be read-only by default; write access requires a human gate; all connector calls must be idempotent

D) Each connector must have its own subagent; connectors should not share authentication tokens; connector errors should pause the loop for human review

✔ Correct Answer: B) Fewer focused tools beat many overlapping ones (model loses track of which fits); writes must be safe to repeat (a retried create makes a duplicate); errors must say what to do next (the error message is the input to the next beat)

Explanation: The three connector rules for loops: (1) fewer focused tools, (2) writes must be safe to repeat (prefer update-or-create over blind creates), (3) errors must say what to do next. In a loop, bad tool picks cost a beat every time, forever.

Q27. Why does the maker-checker split cost more tokens, and when does the course say you should NOT use it?

A) The checker always uses a frontier model which is more expensive; skip it for tasks taking less than 5 minutes

B) Each subagent runs its own model and tools, so the split costs more — but skip it for throwaway, read-only chores where a second opinion doesn't matter. Spend it where a second opinion matters: anything the loop will commit while you are away

C) The checker duplicates all file reads, doubling I/O cost; skip it when working with large files over 1MB

D) Multiple subagents compete for the same context window slots, reducing quality; skip it when the main agent is confident

✔ Correct Answer: B) Each subagent runs its own model and tools, so the split costs more — but skip it for throwaway, read-only chores where a second opinion doesn't matter. Spend it where a second opinion matters: anything the loop will commit while you are away

Explanation: 'Each subagent runs its own model and tools, so the maker-checker split really does cost more tokens... Spend it where a second opinion matters, meaning anything the loop will commit while you are away. Skip it for throwaway, read-only chores.'

Q28. What is a 'dynamic workflow' in Claude Code, and why is the course careful to say it is NOT a loop?

A) A dynamic workflow is a multi-step process that runs indefinitely; it is not a loop because it lacks a heartbeat

B) A dynamic workflow runs once when started and forgets everything when it ends — it has no heartbeat and no spine. It is the BODY of a single beat, not a loop. The loop = heartbeat fires the beat + workflow is the body + progress file is the spine between firings

C) A dynamic workflow uses graph-based orchestration instead of sequential steps; it is not a loop because it can branch

D) A dynamic workflow is a cloud-managed process; it is not a loop because it cannot be interrupted

✔ Correct Answer: B) A dynamic workflow runs once when started and forgets everything when it ends — it has no heartbeat and no spine. It is the BODY of a single beat, not a loop. The loop = heartbeat fires the beat + workflow is the body + progress file is the spine between firings

Explanation: 'A workflow is the body of one beat, not the loop... A dynamic workflow runs once, when you start it, and forgets everything when it ends. It has no heartbeat and no spine.' The memory aid: workflow = engine, Routine = turns the key, progress.md = carries information between trips.

Q29. A verification skill has four possible 'homes.' What is the 'graduation rule' for moving a check from standalone to embedded or chained?

A) Move it when the check passes with 95% success rate across 10 runs

B) The signal is catching yourself running it after every change — at that point it has earned a permanent home. And wait before adding a PR-wide gate while the chain is still changing, because changes to a team-gate are visible to everyone

C) Move it after exactly one month of manual use to ensure stability

D) Move it as soon as it is written, because manual execution creates inconsistency

✔ Correct Answer: B) The signal is catching yourself running it after every change — at that point it has earned a permanent home. And wait before adding a PR-wide gate while the chain is still changing, because changes to a team-gate are visible to everyone

Explanation: 'Do not start at home four. The signal that a check is ready to leave the standalone home is that you catch yourself running it after every change. At that point it has earned a permanent home.' Same trust ladder as a loop itself.

Q30. What is the key difference between a check embedded in a skill and a check chained after a skill?

A) Embedded checks run in parallel; chained checks run sequentially

B) Embedded appends the check to the end of the producing skill (works only on skills you can edit; built-in skills get overwritten on update, silently removing the check). Chained has one skill call another at its end — use a thin wrapper for skills you cannot modify

C) Embedded checks use the same model as the producer; chained checks always use Haiku

D) Embedded checks are temporary; chained checks persist across sessions

✔ Correct Answer: B) Embedded appends the check to the end of the producing skill (works only on skills you can edit; built-in skills get overwritten on update, silently removing the check). Chained has one skill call

another at its end — use a thin wrapper for skills you cannot modify

Explanation: 'Embedding only works on skills you can edit. Built-in skills and plugin-managed skills get overwritten on update, so an appended check silently vanishes. For those, chain instead: write a thin wrapper skill that invokes the original, then invokes your check.'

Q31. The course describes the layer of mechanical checks between 'what tools already include' and 'what a reviewer grades against.' What is this layer and why is it valuable?

A) The model evaluation layer — automated benchmarks that assess output quality

B) Project-specific mechanical rules only you can write (e.g., 'no migration that drops a column without a backfill step') — once written, they become commands a loop can prove rather than claims a rubric must judge. Every rule moved here turns a claim into a proof

C) The linter configuration layer — ESLint and Prettier rules customized for the project

D) The integration test layer — end-to-end tests that verify the full system behavior

✔ Correct Answer: B) Project-specific mechanical rules only you can write (e.g., 'no migration that drops a column without a backfill step') — once written, they become commands a loop can prove rather than claims a rubric must judge. Every rule moved here turns a claim into a proof

Explanation: 'Between the mechanical checks the tools already include and the rubric a reviewer grades against, there is a layer of mechanical checks only you can write... Every rule you move from the rubric down to this rung turns a claim into a proof.'

Q32. The course describes two layers of state in the spine. What are they and what is the key difference in how each is used?

A) The model layer (in-context memory) and the disk layer (external files) — model layer for current beat, disk layer for between beats

B) The rules file (CLAUDE.md/AGENTS.md) — steady habits read on every run and kept short because you pay on every beat; and the progress file — records what was tried, what passed, what is open, updated every run. The rules file is the front of the intern's diary; the progress file is the back

C) The input layer (what the loop reads) and the output layer (what the loop writes) — managed separately for security

D) The fast layer (in-memory cache) and the slow layer (database) — fast for within-beat, slow for between-beat persistence

✔ Correct Answer: B) The rules file (CLAUDE.md/AGENTS.md) — steady habits read on every run and kept short because you pay on every beat; and the progress file — records what was tried, what passed, what is open, updated every run. The rules file is the front of the intern's diary; the progress file is the back

Explanation: The intern's diary: front = rules file (durable lessons, read every run), back = progress file (checkpoints, updated every run). 'An intern without the diary learns the same corrections again and redoes yesterday's work forever.'

Q33. The Paper Watch project proves the spine concept with one destructive command. What happens when you run 'rm progress.md' and ask the same question again?

A) The loop throws an error because the required state file is missing

B) Every paper returns as 'new' because deleting the spine made the loop forget everything — 'no spine, no loop, in one command'

C) The loop regenerates the progress file from the conversation history

D) The loop pauses and asks for confirmation before proceeding without the state file

✔ Correct Answer: B) Every paper returns as 'new' because deleting the spine made the loop forget everything — 'no spine, no loop, in one command'

Explanation: 'Delete the memory and ask once more: rm progress.md, show me what's new... Every paper returns as new. You just made the loop forget everything: no spine, no loop, in one command.'

Q34. Anthropic's production experience led to a 'simplest of all' memory design. What did they settle on after trying alternatives?

A) A vector database with semantic search for memory retrieval across runs

B) Model memory as a plain file system — markdown files in folders, searched with ordinary tools like grep and shell commands rather than a special memory API. The spine in this course is plain markdown on disk

C) A graph database storing relationships between facts learned across sessions

D) A Redis cache with TTL-based expiration for recent context and an S3 archive for older memories

✔ **Correct Answer: B) Model memory as a plain file system — markdown files in folders, searched with ordinary tools like grep and shell commands rather than a special memory API. The spine in this course is plain markdown on disk**

Explanation: 'The current best practice is the simplest of all: model memory as a plain file system. Markdown files in folders. Let the agent search them with the ordinary tools it already knows, like grep and shell commands.' This is where Anthropic's own memory work landed.

Q35. The course describes a 'hill-climbing loop.' What does its output consist of, and how does it differ from the working loops?

A) Its output is completed tasks — it is a faster version of the standard working loop

B) Its output is NOT work — it is improvements to the system that does the work. It reads traces of past runs, finds patterns of mistakes, and proposes changes to the prompt, tools, checker rules, or rules file

C) Its output is evaluation metrics — it grades the quality of working loop outputs

D) Its output is new connectors — it discovers and configures MCP servers the loop might need

✔ **Correct Answer: B) Its output is NOT work — it is improvements to the system that does the work. It reads traces of past runs, finds patterns of mistakes, and proposes changes to the prompt, tools, checker rules, or rules file**

Explanation: 'The loop edits itself.' vs standard loops that do the work. 'Its output is not work. Its output is improvements to the system that does the work... the loop writes the lesson into the rules file, so the fix stays for every future run.'

Q36. What is 'dreaming' as Anthropic's applied AI team defines it, and what is its weekly (not daily) cadence justified by?

A) Dreaming is a real-time learning process that updates model weights during idle periods — weekly cadence because weight updates take time

B) Dreaming is an out-of-band improvement loop that runs while working agents rest — it reads batches of run transcripts, finds patterns repeating across sessions (one mistake is noise, the same mistake three times is a missing lesson), and proposes changes to the memory store. Weekly not daily because it needs a batch of runs to find patterns; daily is too often to see them

C) Dreaming is an automated backup process that archives completed run transcripts — weekly cadence to minimize storage costs

D) Dreaming is a model fine-tuning process that specializes the agent for your project — weekly cadence because training runs require significant compute

✔ **Correct Answer: B) Dreaming is an out-of-band improvement loop that runs while working agents rest — it reads batches of run transcripts, finds patterns repeating across sessions (one mistake is noise, the same mistake three times is a missing lesson), and proposes changes to the memory store. Weekly not daily because it needs a batch of runs to find patterns; daily is too often to see them**

Explanation: 'A weekly cloud Routine... is the right cadence. Daily is too often to see patterns, and it also multiplies cost for nothing.' The dreaming loop reads batches, finds cross-session patterns, proposes changes as PRs — the human approves or closes.

Q37. The course names two failure modes of repeated memory rewriting from research. What are they and what is the fix?

A) Memory bloat (files grow too large) and memory corruption (incorrect facts overwrite correct ones) — fix: delete and rebuild from scratch periodically

B) Brevity bias (rewrites keep the general point, drop the specifics — 'check the response payload' becomes 'handle errors') and context collapse (each rewrite is a lossy copy until a detailed playbook degrades to a vague paragraph) — fix: propose small diffs, never full rewrites; git tracks shrinking files where you can refuse them

C) Memory staleness (outdated facts persist) and memory conflicts (two agents write contradictory facts) — fix: timestamp all memories and use conflict resolution rules

D) Memory fragmentation (related facts spread across files) and memory duplication (same fact stored multiple times) — fix: a weekly consolidation pass that merges related entries

✔ **Correct Answer: B) Brevity bias (rewrites keep the general point, drop the specifics — 'check the response payload' becomes 'handle errors') and context collapse (each rewrite is a lossy copy until a detailed playbook degrades to a vague paragraph) — fix: propose small diffs, never full rewrites; git tracks shrinking files where you can refuse them**

Explanation: From the Stanford/SambaNova/Berkeley paper: 'brevity bias, where a rewrite keeps the general point and drops the specifics... and context collapse, where each full rewrite is a lossy copy of the last.' Fix: 'propose small diffs, never full rewrites.'

Q38. What is 'memory poisoning' in the context of the dreaming loop, and what two rules does the course say defend against it?

A) When a loop writes incorrect facts from hallucinations into the rules file — defended by human review of all memory writes and automated fact-checking

B) An instruction planted in one run's input (e.g., an issue body or PR description written by outsiders) gets written into memory and steers every later run long after the original attack is gone — defended by: evidence always (proposals must cite the source runs) and the human gate always (rule changes that skip review are what an attacker most wants)

C) When two parallel loops write conflicting information to the same memory file — defended by file locking and conflict resolution

D) When old memories gradually corrupt new ones through repeated consolidation — defended by TTL expiration and versioned memory files

✔ Correct Answer: B) An instruction planted in one run's input (e.g., an issue body or PR description written by outsiders) gets written into memory and steers every later run long after the original attack is gone — defended by: evidence always (proposals must cite the source runs) and the human gate always (rule changes that skip review are what an attacker most wants)

Explanation: 'Memory poisoning: an instruction planted in one run's input gets written into memory and steers every later run, long after the original attack is gone. A dreaming pass is the exact machine that could turn a one-time injection into a permanent rule.' Two defenses: evidence always, human gate always.

Q39. The course warns about one absolute rule for CLAUDE.md vs auto-managed memory notes. What is it?

A) CLAUDE.md should only contain technical commands; personal preferences go in auto-managed notes

B) A rule you never want touched goes in CLAUDE.md — the file only YOU control. Auto-managed notes are Claude's notebook and may be rewritten or deleted; CLAUDE.md is yours and the Auto Dream cleaner will not touch it

C) CLAUDE.md is read every run and has a token cost; keep it under 500 tokens and use auto-managed notes for everything else

D) Auto Dream can only modify CLAUDE.md; it cannot touch other memory files in the repository

✔ Correct Answer: B) A rule you never want touched goes in CLAUDE.md — the file only YOU control. Auto-managed notes are Claude's notebook and may be rewritten or deleted; CLAUDE.md is yours and the Auto Dream cleaner will not touch it

Explanation: 'The cleaner trusts newer evidence over older notes, so it may rewrite or delete a note even if you wrote that note by hand. So keep this rule: a rule you never want touched goes in CLAUDE.md, the file only you control.'

Q40. What is the one honest limit on dreaming that the course names, and what does it mean practically?

A) Dreaming only works during business hours — the Anthropic servers that power it run reduced capacity at night

B) A dream needs material — a loop with three runs of history or a throwaway project has no patterns to find, and the pass will produce plausible-sounding lessons from noise. Dream over loops with real mileage

C) Dreaming can only analyze text outputs, not code diffs or binary files

D) Each dreaming pass can review at most 50 run transcripts before it hits context limits

✔ Correct Answer: B) A dream needs material — a loop with three runs of history or a throwaway project has no patterns to find, and the pass will produce plausible-sounding lessons from noise. Dream over loops with real mileage

Explanation: 'One more honest limit: a dream needs material. A loop with three runs of history, or a throwaway project, has no patterns to find, and the pass will produce plausible-sounding lessons from noise. Dream over loops with real mileage.'

Q41. The minimum safe loop checklist has seven items. Which item is described as 'the one part people forget' and why?

A) The limit (max tries) — because people assume the success condition will always be met

B) The state file (spine) — because without it the loop has no memory between runs and repeats its first step forever, but it is easy to skip when focused on the working parts

C) The human gate — because automated systems feel safer without human interruption

D) The log or notification — because people assume they will notice failures without alerts

✔ Correct Answer: B) The state file (spine) — because without it the loop has no memory between runs and repeats its first step forever, but it is easy to skip when focused on the working parts

Explanation: From Concept 2: 'And the sixth, the one beginners skip... State/memory, the spine.' The minimum safe loop checklist item: 'State file: the spine, so it remembers between runs.' Without it, no loop — just the same first step repeated.

Q42. In the morning-triage loop from Part 5, what is the EXACT rule the reviewer uses to decide between PASS and FAIL?

A) The reviewer checks if the code compiles and all tests pass; any other concerns result in PASS

B) PASS requires tests to actually pass AND the change must do only what was asked — 'a change that only looks fine is not a PASS.' FAIL is given for specific reasons. The reviewer never edits files — it is read-only

C) The reviewer grades on a 1-10 scale and PASS is anything above 7

D) PASS is automatic if no security vulnerabilities are detected by the linter

✔ Correct Answer: B) PASS requires tests to actually pass AND the change must do only what was asked — 'a change that only looks fine is not a PASS.' FAIL is given for specific reasons. The reviewer never edits files — it is read-only

Explanation: From the reviewer agent definition: 'A change that only looks fine is not a PASS. The tests must actually pass, and the change must do only what was asked.' The reviewer must run tests itself and read output — not trust claims.

Q43. The Part 5 loop has a rule: 'When in doubt, escalate. A flagged item a human checks is always safer than a wrong fix shipped while no one was watching.' What SPECIFIC action does the loop take instead of opening a PR on risky changes?

A) It sends an email to the team distribution list and waits for a reply before proceeding

B) It adds a short entry to the 'Open / needs a human' section of progress.md with what was tried and why it stopped — it does NOT open a pull request

C) It opens a draft PR marked WIP and adds a 'needs-review' label for a human to complete

D) It reverts all changes in the worktree and logs the failure to the CI system

✔ Correct Answer: B) It adds a short entry to the 'Open / needs a human' section of progress.md with what was tried and why it stopped — it does NOT open a pull request

Explanation: Directly from the daily-triage skill: 'FAIL, or the change touches anything risky: do NOT open a pull request. Add a short entry to the Open/needs a human section of progress.md. Say what you tried and why you stopped.'

Q44. The Part 5 loop limits itself to a maximum of 5 pull requests per run. Why is a per-run limit on PR count important even when each PR passed the reviewer?

A) GitHub's API rate limits prevent more than 5 PRs from being opened per hour

B) The limit prevents a runaway loop from flooding the repository with changes when the stopping condition is unclear or when the loop misunderstands the work — it bounds the blast radius of any single automated run

C) Code review tools cannot process more than 5 PRs simultaneously without quality degradation

D) The daily Routine cap prevents more than 5 complete loop cycles from running anyway

✔ Correct Answer: B) The limit prevents a runaway loop from flooding the repository with changes when the stopping condition is unclear or when the loop misunderstands the work — it bounds the blast radius of any single automated run

Explanation: From the skill: 'Never open more than 5 pull requests in one run.' This is blast-radius management — even with a passing reviewer, a confused loop could generate many spurious PRs without this cap.

Q45. The Part 5 loop uses the claude/ branch prefix. What is the DUAL purpose this prefix serves?

A) It organizes branches alphabetically in the GitHub UI and marks them for automatic garbage collection after 30 days

B) It is a guardrail that limits where the unattended agent can push (only claude/* branches by default), AND it signals to humans which branches were opened by automation vs. manually — both safety and transparency

C) It enables the Claude GitHub App to automatically detect and process these branches

D) It prevents merge conflicts by ensuring all automated branches share a common base

✔ Correct Answer: B) It is a guardrail that limits where the unattended agent can push (only claude/* branches by default), AND it signals to humans which branches were opened by automation vs. manually — both safety

and transparency

Explanation: From Appendix A2: 'By default Claude can push only to branches whose names start with claude/.' The prefix is the permission boundary AND the human-readable signal that automation created this branch. 'Only claude/ branches' is the rule that keeps automation from touching main.*

Q46. Andrew Ng describes three feedback loops in product development. What are the three, and what does the course say only you can provide for the outer two?

A) Design loop (days), build loop (hours), test loop (minutes) — only humans can access user research data

B) Coding loop (minutes, runs by itself), feedback loop (hours, you try the output and update instructions), outside loop (days, real people use it and show you what to fix next) — the outer two need you because you have context advantage: who will use this, what they really need, what 'good' feels like

C) Specification loop (weeks), implementation loop (days), verification loop (hours) — only humans can write specifications

D) Discovery loop (months), delivery loop (weeks), deployment loop (days) — only humans can make business decisions

✔ Correct Answer: B) Coding loop (minutes, runs by itself), feedback loop (hours, you try the output and update instructions), outside loop (days, real people use it and show you what to fix next) — the outer two need you because you have context advantage: who will use this, what they really need, what 'good' feels like

Explanation: Andrew Ng's three loops: coding (minutes), developer feedback (hours), external feedback (days). 'You know things it does not: who will use this, what they really need, and what good feels like. As long as you know something the agent does not, you stay in the loop to tell it. Ng calls this your context advantage.'

Q47. The course names 'AI gravity' from MIT Sloan's Eric So. What is it, and why does a loop make it stronger than ordinary AI use?

A) The tendency for AI models to gravitate toward the most common answer — loops amplify this by repeating the same prompt multiple times

B) The steady pull to let AI do more and more of your thinking — a loop makes it stronger because it runs while you sleep, so the gap between what ships and what you understand can grow even on days you touch nothing

C) The computational gravity that makes larger models more capable — loops amplify this by chaining multiple model calls

D) The tendency for AI outputs to cluster around median quality — loops amplify this through repeated averaging of results

✔ Correct Answer: B) The steady pull to let AI do more and more of your thinking — a loop makes it stronger because it runs while you sleep, so the gap between what ships and what you understand can grow even on days you touch nothing

Explanation: 'AI gravity: the steady pull to let AI do more and more of your thinking. A loop makes that pull stronger, because it runs while you sleep.' Intent slips from precise conditions to 'just keep it working'; accountability slips from reading diffs to trusting green checkmarks.

Q48. The course describes 'in the loop,' 'on the loop,' and 'out of the loop.' What is the CORRECT mapping to the Routine concepts built in the course?

A) In the loop = cloud Routines; on the loop = in-session loops; out of the loop = scheduled tasks

B) In the loop = person approves every action (prompting turn by turn, plan mode, merge at human gate). On the loop = system acts on its own, person watches and can step in (Routine pushing to claude/ while you review each morning). Out of the loop = nobody watching — never acceptable for writes

C) In the loop = human writes all prompts; on the loop = human reviews all outputs; out of the loop = full automation

D) In the loop = Claude Code; on the loop = OpenCode; out of the loop = unmonitored cron jobs

✔ Correct Answer: B) In the loop = person approves every action (prompting turn by turn, plan mode, merge at human gate). On the loop = system acts on its own, person watches and can step in (Routine pushing to claude/ while you review each morning). Out of the loop = nobody watching — never acceptable for writes

Explanation: The course maps explicitly: in-the-loop = you approve each action. On-the-loop = system acts, person watches and can stop it (morning-triage Routine with claude/ branches). Out-of-the-loop = 'Nowhere, on purpose. In this course, this is not a third option. It is the failure mode.'

Q49. The course says the morning-triage loop is 'on-the-loop for safe fixes and drops back in-the-loop for anything risky.' The checker ladder determines this mix. What is the rule?

A) All automated actions require in-the-loop approval until the loop has run successfully 100 times

B) The weaker the checker, the more actions must move from on-the-loop back to in-the-loop — a passing test earns autonomy, a rubric score does not

C) In-the-loop is required for all external actions (PRs, Slack posts); on-the-loop is acceptable only for local file edits

D) The mix is determined by the model used — frontier models earn on-the-loop status, cheaper models require in-the-loop

✔ Correct Answer: B) The weaker the checker, the more actions must move from on-the-loop back to in-the-loop — a passing test earns autonomy, a rubric score does not

Explanation: 'The weaker the checker, the more actions must move from on-the-loop back to in-the-loop. A passing test earns autonomy. A rubric score does not.' This is the checker ladder applied to the human gate setting.

Q50. How does the dogfooding section say 'out of the loop' happens in practice, and what is 'the defense'?

A) Out of the loop happens when the loop has a bug — the defense is comprehensive test coverage

B) Nobody designs an out-of-the-loop system on purpose. It happens by drift — stop reading the diffs, trust the green checkmarks, skip the weekly read of what shipped. The defense is the dogfooding rule: put the human where a wrong automatic move is costly and hard to reverse, and check that the human is ACTUALLY still there

C) Out of the loop happens when the Routine's daily cap is exceeded — the defense is monitoring the usage page

D) Out of the loop happens when the reviewer agent is misconfigured — the defense is testing the reviewer separately

✔ Correct Answer: B) Nobody designs an out-of-the-loop system on purpose. It happens by drift — stop reading the diffs, trust the green checkmarks, skip the weekly read of what shipped. The defense is the dogfooding rule: put the human where a wrong automatic move is costly and hard to reverse, and check that the human is ACTUALLY still there

Explanation: 'Stop reading the diffs, trust the green checkmarks, skip the weekly read of what shipped, and a loop you designed as on-the-loop quietly becomes out-of-the-loop, with no design change at all.' The defense: Concept 15's weekly habit — read what shipped, check that understanding kept up.

Q51. What is the token cost example the course gives for a loop running every 5 minutes vs. once an hour, and what is the key takeaway?

A) 5-minute loop: ~$5/month; hourly loop: ~$0.50/month — the 10x cost difference makes frequency the biggest lever

B) One beat at Sonnet pricing costs ~$0.20. Five beats/day for 20 days ≈ $20/month. The same loop every 5 minutes performs 100x+ beats, potentially exceeding $1,000/month. 'Frequency, not the command name, drives this increase' — frequency and retry rate determine how many beats you pay for

C) 5-minute loops are throttled after 50 runs/hour by the API — the cost stays constant due to automatic rate limiting

D) Token cost doubles for every model generation — the 10x cost difference between Haiku and Sonnet is the primary lever, not frequency

✔ Correct Answer: B) One beat at Sonnet pricing costs ~$0.20. Five beats/day for 20 days ≈ $20/month. The same loop every 5 minutes performs 100x+ beats, potentially exceeding $1,000/month. 'Frequency, not the command name, drives this increase' — frequency and retry rate determine how many beats you pay for

Explanation: 'A loop running every five minutes, all day and all night, performs more than one hundred times as many beats. Its monthly cost can easily exceed $1,000.' 'Frequency, not the command name, drives this increase.' This is Concept 13's central lesson.

Q52. The course says 'a good spine is also a cost lever.' How does a well-kept memory store reduce token costs?

A) A well-organized spine compresses redundant information, reducing the tokens needed to represent state

B) An agent with a well-kept memory store does the task better the second time (the lesson from the first attempt is already on disk), leading to fewer retries — fewer retries mean fewer tokens. 'The loop gets more accurate and cheaper at the same time'

C) A spine in a separate repository reduces context window usage, leaving more tokens for the actual task

D) A compressed spine file format allows faster read times, reducing the latency cost of each beat

✔ Correct Answer: B) An agent with a well-kept memory store does the task better the second time (the lesson from the first attempt is already on disk), leading to fewer retries — fewer retries mean fewer tokens. 'The loop gets more accurate and cheaper at the same time'

Explanation: Anthropic's production finding: 'An agent with a well-kept memory store does the task better the second time, because the lesson from the first attempt is already on disk. Better first attempts mean fewer retries, and fewer retries mean

fewer tokens.'

Q53. In the context of many loops running in production, the course describes 'the counting question.' What are the hard questions it names, and what field does it point toward?

A) How many tokens do these loops consume, what are the API costs, and who pays — pointing toward budget management

B) How many loops does this team run? What can each one touch? Whose identity does each one act under? Who approved them? — 'That is no longer loop engineering. It is workforce management. It is exactly where this book's Human-Agent Teams material and the Digital-FTE idea pick up'

C) How many parallel runs occur simultaneously, what are the conflict rates, and how are they resolved — pointing toward distributed systems design

D) How many humans review loop outputs per day, what is their accuracy rate, and how do we reduce their workload — pointing toward automation percentage optimization

✔ Correct Answer: B) How many loops does this team run? What can each one touch? Whose identity does each one act under? Who approved them? — 'That is no longer loop engineering. It is workforce management. It is exactly where this book's Human-Agent Teams material and the Digital-FTE idea pick up'

Explanation: 'How many loops does this team run? What can each one touch? Whose identity does each one act under? Who approved them? That is no longer loop engineering. It is workforce management.'

Q54. The course describes four guardrails for shared memory in a multi-loop fleet. What are they?

A) Encryption, access logging, anomaly detection, and automatic rollback

B) Versioning (every change recorded with what/which run/who); conflict checks before writing (check if file changed while drafting); permissions by level (ordinary agents read-only for org-wide rules, changes via review); portability (plain open format behind a clean interface)

C) File locking, transaction queuing, priority scheduling, and cache invalidation

D) Backup copies, read-only snapshots, write throttling, and human approval chains

✔ Correct Answer: B) Versioning (every change recorded with what/which run/who); conflict checks before writing (check if file changed while drafting); permissions by level (ordinary agents read-only for org-wide rules, changes via review); portability (plain open format behind a clean interface)

Explanation: The four guardrails from Anthropic's production team: (1) versioning, (2) conflict checks before writing, (3) permissions by level, (4) portability. 'A memory a fleet depends on is production data, and it deserves production discipline.'

Q55. The dogfooding section describes two loops that keep the book running. What is the deciding factor for why Loop 1 (feedback loop) has a human gate but Loop 2 (What's New loop) has none?

A) Loop 1 involves external users so requires compliance review; Loop 2 is internal so can be automated

B) The deciding factor is the cost of a wrong move — a bad edit to a lesson is expensive and hard to undo (human gate required); a clumsy changelog line is one revert away from fixed (no gate needed). 'Put the human where a wrong automatic move would be costly and hard to reverse'

C) Loop 1 uses Claude Code which requires human approval; Loop 2 uses OpenCode which is fully autonomous

D) Loop 1 processes sensitive reader data requiring GDPR compliance; Loop 2 only processes public repository data

✔ Correct Answer: B) The deciding factor is the cost of a wrong move — a bad edit to a lesson is expensive and hard to undo (human gate required); a clumsy changelog line is one revert away from fixed (no gate needed). 'Put the human where a wrong automatic move would be costly and hard to reverse'

Explanation: 'A bad edit to a lesson is expensive and hard to undo. A clumsy changelog line is one revert away from fixed. So the rule... is short: put the human where a wrong automatic move would be costly and hard to reverse, and leave the human out everywhere else.'

Q56. What is the key difference between a 'Remote' and 'Local' option in the Claude Code Desktop app's New Routine dialog?

A) Remote Routines can access external APIs; Local Routines can only read files

B) Remote = a cloud Routine running on Anthropic's servers (laptop-closed guarantee). Local = a Desktop scheduled task running on YOUR machine against real files including unsaved changes — only while your machine is on. 'A good first step is to prove a prompt as a Desktop task, then move it to a cloud Routine'

C) Remote Routines run every hour minimum; Local Routines can run every minute

D) Remote Routines require paid plans; Local Routines are available on free plans

✔ **Correct Answer: B) Remote = a cloud Routine running on Anthropic's servers (laptop-closed guarantee). Local = a Desktop scheduled task running on YOUR machine against real files including unsaved changes — only while your machine is on. 'A good first step is to prove a prompt as a Desktop task, then move it to a cloud Routine'**

Explanation: From Appendix A1: 'Remote creates a cloud routine... Local creates a Desktop scheduled task: a different feature that runs on your machine, against your real files including unsaved changes, only while your machine is on.'

Q57. The appendix says the 'all connectors included' default in Routines is 'the one default worth changing every time.' Why?

A) Including all connectors makes each run slower because the model must evaluate which connector to use

B) All connected connectors are included by default with write access, so an unattended agent can act in every tool you have linked without asking. Remove everything the Routine does not need before saving — this is both a safety AND functional scoping rule

C) Connector overhead increases token usage by approximately 30% per connector included

D) Multiple connectors can interfere with each other's authentication, causing sporadic failures

✔ **Correct Answer: B) All connected connectors are included by default with write access, so an unattended agent can act in every tool you have linked without asking. Remove everything the Routine does not need before saving — this is both a safety AND functional scoping rule**

Explanation: 'All of your connected claude.ai connectors are included by default. Claude can use every tool on them, including writes, without asking. Remove everything the routine does not need before you save.'

Q58. Why does putting secrets in a .env file fail for cloud Routines, and what is the correct approach?

A) .env files use a format the cloud environment cannot parse; use JSON config files instead

B) .env is gitignored, gitignored files never reach GitHub, the cloud clone never contains them, and the Routine finds no credentials and fails. Correct: put every key in the environment's variables panel AND add a line to the prompt saying 'credentials are available as environment variables; do not look for a .env file'

C) .env files are insecure because they travel through version control; use encrypted secrets managers

D) Cloud Routines run as a different user than the local environment, so .env files are inaccessible due to file permissions

✔ **Correct Answer: B) .env is gitignored, gitignored files never reach GitHub, the cloud clone never contains them, and the Routine finds no credentials and fails. Correct: put every key in the environment's variables panel AND add a line to the prompt saying 'credentials are available as environment variables; do not look for a .env file'**

Explanation: Appendix A4: '.env is gitignored, gitignored files never reach GitHub, and the cloud clone therefore never contains them. The routine fires, finds nothing, and fails (or worse, improvises). Put every key in the environment's variables panel.'

Q59. What specific danger does 'a retrying webhook' create in the context of API-triggered Routines?

A) Retrying webhooks can cause authentication token expiration, locking the Routine out of connected services

B) The /fire endpoint has no built-in deduplication, and webhook senders retry by default — a retried alert is a duplicate run, and a misconfigured alert retrying all night uses the whole day's cap before you wake (and with metered extra usage on, stops being a ceiling and becomes a bill)

C) Retrying webhooks cause race conditions when multiple Routine instances try to modify the same files simultaneously

D) Webhook retries trigger GitHub's abuse detection and temporarily block the repository from receiving further events

✔ **Correct Answer: B) The /fire endpoint has no built-in deduplication, and webhook senders retry by default — a retried alert is a duplicate run, and a misconfigured alert retrying all night uses the whole day's cap before you wake (and with metered extra usage on, stops being a ceiling and becomes a bill)**

Explanation: 'The /fire endpoint has no built-in deduplication, and webhook senders retry by default. So a retried alert is a duplicate run, and a misconfigured alert that retries all night is a loop you never designed.'

Q60. The appendix warns about 'matches regex' in GitHub trigger filters. What is the trap and how do you avoid it?

A) Regex patterns are case-sensitive by default, so 'hotfix' won't match 'HOTFIX' — add the (?i) flag

B) 'matches regex' tests the ENTIRE field, not a substring. 'hotfix' only matches a title that is literally 'hotfix' (nothing else). Write '.*hotfix.*' for a substring match, or just use 'contains' instead

C) Regex patterns with special characters must be double-escaped because the filter parser is not regex-aware

D) GitHub trigger regex filters are evaluated on the server side and may have different regex flavors than expected

✔ Correct Answer: B) 'matches regex' tests the ENTIRE field, not a substring. 'hotfix' only matches a title that is literally 'hotfix' (nothing else). Write '.*hotfix.*' for a substring match, or just use 'contains' instead

*Explanation: Appendix A3: 'One important detail: matches regex tests the whole field, not a part of it. So hotfix matches only a title that is exactly hotfix. Write .*hotfix.*, or just use contains.'*

Q61. What does 'green status' mean for a Routine run, and why is the appendix emphatic that you must open the transcript anyway?

A) Green means the Routine completed without exceeding token limits — open the transcript to see what was accomplished

B) Green status means the platform completed the session without an infrastructure error — it does NOT prove the task succeeded. Blocked network requests, missing connector tools, and plain task failures all live in the transcript, not in the status column. 'Open the run and read it'

C) Green means all tests in the repository passed during the run — still open the transcript for task-specific results

D) Green means the Routine will run again at the scheduled time — open the transcript only if you want to see intermediate results

✔ Correct Answer: B) Green status means the platform completed the session without an infrastructure error — it does NOT prove the task succeeded. Blocked network requests, missing connector tools, and plain task failures all live in the transcript, not in the status column. 'Open the run and read it'

Explanation: Appendix A5: 'Green means the platform completed the session. It does not prove that the requested work succeeded.' This is one of the most important practical warnings in the appendix.

Q62. The two-Routine gate pattern solves the 'no mid-run approval' limitation of cloud Routines. How does it work?

A) Two Routines run in parallel — one drafts, one reviews simultaneously, and both must complete before any action is taken

B) Routine A drafts the work and posts it somewhere reviewable (claude/ branch, Slack message, draft email). A human reads it and approves. That approval fires Routine B through its API trigger, and Routine B does the action

C) The first Routine creates a GitHub issue requesting approval, and the second Routine is triggered when the issue is labeled 'approved'

D) Two Routines share a state file — Routine A writes 'READY' to signal Routine B to proceed, mimicking a mid-run approval

✔ Correct Answer: B) Routine A drafts the work and posts it somewhere reviewable (claude/ branch, Slack message, draft email). A human reads it and approves. That approval fires Routine B through its API trigger, and Routine B does the action

Explanation: Appendix A4: 'Build the gate between routines, not inside one... Routine A drafts the work and posts it somewhere you can review... A human reads it and approves. That approval fires Routine B through its API trigger, and Routine B does the action.'

Q63. What does the course mean by 'every part of a loop as a standing permission,' and what examples illustrate this?

A) Every loop part requires explicit written approval in the CLAUDE.md file before it can execute

B) A schedule is permission to act while you sleep; a connector is standing access to a real system; a subagent acts under a borrowed identity. 'Give each loop only what its job needs' — the claude/ prefix, the short connector list, and the read-only checker are all this same idea

C) All loop parts inherit permissions from the user account that created the loop

D) Loop permissions are defined in a separate permissions manifest file that must be committed to the repository

✔ Correct Answer: B) A schedule is permission to act while you sleep; a connector is standing access to a real system; a subagent acts under a borrowed identity. 'Give each loop only what its job needs' — the claude/ prefix, the short connector list, and the read-only checker are all this same idea

Explanation: 'Think of every part of a loop as a standing permission. A schedule is permission to act while you sleep. A connector is standing access to a real system. A subagent acts under a borrowed identity. So give each loop only what its job needs.'

Q64. The course uses the metaphor of 'AI gravity pulling toward out-of-the-loop.' What is 'Concept 15's weekly habit' that acts as the defense?

A) Run a weekly dreaming pass to identify drift in the rules file

B) Read what the loop shipped and check that your understanding kept up with what the loop changed — the habit of actively verifying that you are still the engineer, not just the person who presses go

C) Run the loop in report-only mode every Sunday to review its decisions without committing anything

D) Schedule a weekly meeting with all stakeholders to review automated changes from the past week

✔ Correct Answer: B) Read what the loop shipped and check that your understanding kept up with what the loop changed — the habit of actively verifying that you are still the engineer, not just the person who presses go

Explanation: Concept 15: 'Read what shipped, and check that your understanding kept up with what the loop changed.' This is the defense against AI gravity — active engagement with the loop's output, not passive trust in green checkmarks.

Q65. The course says there is a trap in 'designing the loop keeps you engaged.' What is the trap?

A) Designing loops is so complex that it crowds out time for actual software engineering

B) Designing the loop keeps you engaged when done with care, but lets you stop thinking when done to avoid the work — same action, opposite result. 'The loop cannot tell the difference. You can.' Two people can build the exact same loop and get opposite results

C) Over-designing loops creates brittle systems that break frequently and require constant manual intervention

D) Loop design becomes addictive — teams spend more time building loops than delivering value to users

✔ Correct Answer: B) Designing the loop keeps you engaged when done with care, but lets you stop thinking when done to avoid the work — same action, opposite result. 'The loop cannot tell the difference. You can.' Two people can build the exact same loop and get opposite results

Explanation: 'Designing the loop keeps you engaged when you do it with care. It lets you stop thinking when you do it to avoid the work. Same action, opposite result... Build the loop. But build it like someone who plans to stay the engineer, not just the person who presses go.'

Q66. The course says a verification skill's allowed-tools line 'takes tool names only, not individual commands.' What does this mean for security, and where is the enforcement layer?

A) Allowed-tools is only advisory — the model chooses which tools to use regardless of this setting

B) The allowed-tools line narrows the toolbox to specific tool categories (Read, Bash) but cannot pin the reviewer to just the test and lint commands — that enforcement (command-level restriction) lives in the Harness Engineering course, not this one

C) Allowed-tools accepts both tool names and specific commands — the syntax 'Bash: npm test' restricts to that exact command

D) The allowed-tools restriction is enforced by the Claude API which rejects tool calls outside the allowlist

✔ Correct Answer: B) The allowed-tools line narrows the toolbox to specific tool categories (Read, Bash) but cannot pin the reviewer to just the test and lint commands — that enforcement (command-level restriction) lives in the Harness Engineering course, not this one

Explanation: 'The tools line takes tool names only (Read, Bash), so it cannot pin the reviewer to just the test, lint, and diff commands. For now, the numbered instructions carry that limit. The next course, Harness Engineering, adds the rule that enforces it.'

Q67. The course describes the 'Ralph loop' as a learning tool. What makes it useful pedagogically, and what are its deliberate limitations?

A) The Ralph loop is useful because it is the simplest possible event-driven loop, showing how events trigger actions

B) The Ralph loop is the simplest well-known run-until-done loop — runs the same prompt again and again, each run reading and updating one state file. It keeps only two of three stops (success condition and time cap, no stuck-check), no skill, no separate checker. Its bareness shows clearly why each missing part matters

C) The Ralph loop is pedagogically useful because it uses no AI — demonstrating what loops looked like before AI coding agents

D) The Ralph loop is the production-ready starting template — its limitations are features that prevent common beginner mistakes

✔ Correct Answer: B) The Ralph loop is the simplest well-known run-until-done loop — runs the same prompt again and again, each run reading and updating one state file. It keeps only two of three stops (success

condition and time cap, no stuck-check), no skill, no separate checker. Its bareness shows clearly why each missing part matters

Explanation: 'A Ralph loop with a vague condition wanders until the time cap runs out. The same loop with a condition a command can prove works well.' Its deliberate incompleteness teaches by contrast what each missing part provides.

Q68. The course describes 'context engineering' as distinct from 'loop engineering.' What specific failure does context engineering solve that loop engineering cannot?

A) Context engineering solves hallucination problems by providing more facts; loop engineering cannot add facts

B) Context engineering controls everything the model sees in one turn — strong context can rescue a weak prompt, but no context engineering can rescue a missing checker or a schedule that is still you. Each layer stops a different kind of failure; no single layer can carry the others

C) Context engineering handles long-term memory across sessions; loop engineering only manages within-session state

D) Context engineering is for single-agent use; loop engineering only applies to multi-agent systems

✔ Correct Answer: B) Context engineering controls everything the model sees in one turn — strong context can rescue a weak prompt, but no context engineering can rescue a missing checker or a schedule that is still you. Each layer stops a different kind of failure; no single layer can carry the others

Explanation: 'Strong context can rescue a weak prompt, but no prompt can rescue missing context, a missing checker, or a schedule that is still you. So when you build, a useful self-check is: which of these layers am I still doing by hand?'

Q69. The course mentions that Andrej Karpathy described 'giving success criteria and watching it go' as superior to telling the agent what to do. How does this relate to the stopping condition concept?

A) Karpathy advocates removing all stopping conditions and trusting the model to know when it is done

B) Success criteria ARE stopping conditions — the whole concept of /goal and the conditional loop is operationalizing Karpathy's insight: instead of directing each step, you define what done looks like precisely enough to be tested, and the loop works until that test passes

C) Karpathy's approach applies only to machine learning training loops, not to software development loops

D) Success criteria are different from stopping conditions — Karpathy's criteria are qualitative, while stopping conditions must be quantitative

✔ Correct Answer: B) Success criteria ARE stopping conditions — the whole concept of /goal and the conditional loop is operationalizing Karpathy's insight: instead of directing each step, you define what done looks like precisely enough to be tested, and the loop works until that test passes

Explanation: The course connects this directly: 'Don't tell it what to do, give it success criteria and watch it go.' This is the conditional loop and /goal in practice. 'The loop is only as good as its stopping condition.'

Q70. Why does the course say 'the spec from Spec-Driven Development pays off' specifically for the /goal command and the reviewer agent?

A) Spec-Driven Development generates CLAUDE.md files that Claude Code automatically loads as context

B) A good spec's acceptance criteria are already conditions a command can prove — so it gives you the /goal stopping condition for free. The reviewer grades against the spec's acceptance criteria: 'A vague spec gives you a vague verdict'

C) Specs written in Spec-Driven Development include automated test generation that populates the test suite the reviewer checks

D) Spec-Driven Development produces architecture diagrams that the loop uses to verify structural correctness

✔ Correct Answer: B) A good spec's acceptance criteria are already conditions a command can prove — so it gives you the /goal stopping condition for free. The reviewer grades against the spec's acceptance criteria: 'A vague spec gives you a vague verdict'

Explanation: 'Write conditions a command can prove... This is where the spec from Spec-Driven Development pays off. Its acceptance criteria are already conditions a command can prove, so a good spec gives you the stopping condition for free.'

Q71. The course describes graph engineering as what comes 'after loops.' What is the precise relationship between loop engineering and graph engineering?

A) Graph engineering replaces loop engineering — loops are an outdated paradigm that graphs supersede

B) A graph is loops, composed — remove the loops and the graph is empty boxes. Every stopping condition, checker, spine, and gate from loop engineering is what graph engineering assumes you already know how to build. 'Build your first loop exactly as this course taught. The moment you build your second, the graph course is waiting'

C) Graph engineering handles data processing pipelines; loop engineering handles agent workflows — they are parallel tracks

D) Graph engineering is the theoretical foundation; loop engineering is the practical implementation. You learn graph engineering first, then implement it as loops

✔ Correct Answer: B) A graph is loops, composed — remove the loops and the graph is empty boxes. Every stopping condition, checker, spine, and gate from loop engineering is what graph engineering assumes you already know how to build. 'Build your first loop exactly as this course taught. The moment you build your second, the graph course is waiting'

Explanation: 'A graph is loops, composed. Remove the loops and the graph is empty boxes. Every stopping condition, checker, spine, and gate you built here is what graph engineering assumes you already know how to build.'

Q72. The course describes what loop engineering looks like for non-code work (documents, reports, books). What changes and what stays the same?

A) Everything changes — document loops use completely different tools and cannot use worktrees, skills, or connectors

B) The loop's shape stays the same. What changes is the checker: code has the most honest checkers (tests, linters that prove 'done'), while prose leans on mechanical checks (broken links, banned words, heading levels) plus a reviewer agent with a written rubric. 'Give it a score and a bar: Grade this draft against the rubric. Do not stop below 95' turns a soft judgment into something the loop can act on

C) Document loops cannot run unattended — they always require human review before proceeding

D) The heartbeat changes for documents — only conditional loops work; schedules and events cannot process unstructured content

✔ Correct Answer: B) The loop's shape stays the same. What changes is the checker: code has the most honest checkers (tests, linters that prove 'done'), while prose leans on mechanical checks (broken links, banned words, heading levels) plus a reviewer agent with a written rubric. 'Give it a score and a bar: Grade this draft against the rubric. Do not stop below 95' turns a soft judgment into something the loop can act on

Explanation: 'The loop's shape does not care what the work is.' But: 'Code has the most honest checkers there are, meaning tests and linters, commands that prove done. Prose has no test suite.' The rubric-with-a-bar workaround: 'a score from a model is still a claim, not a proof, and it is weaker than a passing test.'

Q73. The What's New loop in the dogfooding section uses OpenCode, not Claude Code. What does the course say this illustrates about the two tools?

A) OpenCode is better for scheduled tasks because it has no daily run cap

B) It illustrates Concept 3's central argument: the loop shape is the same across tools — 'learn the loop once, and it carries across tools.' The two loops that run the book use different tools (Claude Code and OpenCode) but the same design. 'Only the heartbeat and where the run happened' differed in Part 5's dual implementation

C) OpenCode is used because it supports GitHub Actions natively while Claude Code requires Routines

D) The dogfooding section uses both tools to demonstrate that loop engineering is a multi-tool discipline requiring expertise in both

✔ Correct Answer: B) It illustrates Concept 3's central argument: the loop shape is the same across tools — 'learn the loop once, and it carries across tools.' The two loops that run the book use different tools (Claude Code and OpenCode) but the same design. 'Only the heartbeat and where the run happened' differed in Part 5's dual implementation

Explanation: 'The book does to itself exactly what it is teaching you to do for yourself. They run on two different stacks, which is Concept 3 made real: learn the loop once, and it carries across tools.'

Q74. The course mentions that the 'ultracode' keyword replaced an older trigger word for dynamic workflows. What does this illustrate about the course's advice to 'remember the six-part shape and forget every keystroke'?

A) It shows that Claude Code's API is unstable and should not be used in production

B) It illustrates exactly why the course separates the lasting layer (the six-part shape of a loop) from the mechanical layer (commands, flags, model names). Ultracode replaced an earlier keyword mid-2026 — 'treat each command as a pointer to the live docs, not a fact to memorize'

C) It demonstrates that OpenCode is more stable than Claude Code because OpenCode hasn't changed its core commands

D) It shows that Anthropic deprecates features quickly, so loops should be kept simple to minimize maintenance

✔ **Correct Answer: B) It illustrates exactly why the course separates the lasting layer (the six-part shape of a loop) from the mechanical layer (commands, flags, model names). Ultracode replaced an earlier keyword mid-2026 — 'treat each command as a pointer to the live docs, not a fact to memorize'**

Explanation: 'The lasting layer: the shape of a loop... This is the skill. It stays true after every command below has changed. The mechanical layer: every flag, path, model name, and command. These tools update every week.' Ultracode's rename proves the point.

Q75. The course says 'a memory file should never contain a relative date.' Why, and what does the dreaming loop do to enforce this?

A) Relative dates cause timezone conversion errors when loops run across different geographic regions

B) 'Yesterday we chose Redis' is meaningless in six weeks — every consolidation pass converts 'yesterday' to the actual date, and loops should write absolute dates in the first place. Relative dates are one of the ways memory degrades through context collapse

C) The Anthropic API cannot parse relative date expressions in memory files it processes

D) Relative dates create ordering ambiguity when multiple runs write to the same memory file on the same day

✔ **Correct Answer: B) 'Yesterday we chose Redis' is meaningless in six weeks — every consolidation pass converts 'yesterday' to the actual date, and loops should write absolute dates in the first place. Relative dates are one of the ways memory degrades through context collapse**

Explanation: 'One small habit prevents a whole class of decay: a memory file should never contain a relative date. Yesterday we chose Redis is meaningless in six weeks. Every consolidation pass converts yesterday to the actual date, and your loops should write absolute dates in the first place.'

Q76. The course describes Peter Steinberger's midnight question about 'loops vs graphs.' What is the 'honest version of the slogan' the course says to keep?

A) 'Loop engineering is dead, long live graph engineering' — the slogan is correct and graphs replace loops

B) 'A graph is loops, composed' — the honest version, because 'the slogan is wrong' but 'the pattern it points at is stable.' Steinberger's question points to a real evolution (wiring multiple loops together), but dismissing loops would abandon the building blocks graphs depend on

C) 'Loop engineering and graph engineering are the same thing at different scales' — there is no meaningful distinction

D) 'Graph engineering is optional' — loops are sufficient for all practical use cases and graphs add unnecessary complexity

✔ **Correct Answer: B) 'A graph is loops, composed' — the honest version, because 'the slogan is wrong' but 'the pattern it points at is stable.' Steinberger's question points to a real evolution (wiring multiple loops together), but dismissing loops would abandon the building blocks graphs depend on**

Explanation: 'Keep only the honest version of the slogan, because everything else assumes it: a graph is loops, composed. Remove the loops and the graph is empty boxes.' The slogan 'loop engineering is dead' is wrong — graphs assume you can already build loops.

Q77. The course defines 'unattended' as a key property. What is the precise distinction the course draws about WHERE the timer lives for attended vs. unattended loops?

A) Attended loops use human-readable prompts; unattended loops use machine-optimized configuration

B) An in-session loop keeps its timer in your open session (the process running while your terminal is open) — close the session and it stops. A scheduled task or Routine moves the timer outside the session onto a scheduler that never sleeps. 'Do you want to stay awake between beats?' determines which you need

C) Attended loops run on your hardware; unattended loops always run on cloud infrastructure

D) Attended loops use polling; unattended loops use push notifications from external services

✔ **Correct Answer: B) An in-session loop keeps its timer in your open session (the process running while your terminal is open) — close the session and it stops. A scheduled task or Routine moves the timer outside the session onto a scheduler that never sleeps. 'Do you want to stay awake between beats?' determines which you need**

Explanation: 'An in-session loop keeps its timer in your open session... A scheduled task or Routine moves the timer outside the session, onto a scheduler that never sleeps... Close the session and the thing holding the timer is gone, so the loop stops.'

Q78. The Portfolio project (Concept 5) has a rule it 'will not bend on.' What is it, and what does noticing the urge to break it teach?

A) Never delete files during a build run — the loop must only add and modify, never remove

B) 'Never edit check.py to make it pass' — this is exactly what a loop optimizing for green will reach for. Noticing the urge is the lesson: the measure is the target, and teaching yourself to stop when the loop is about to game its own checker is the real skill

C) Never stop the loop before it reaches its 15-attempt cap — always let it exhaust its limit

D) Never use the reviewer agent on the first attempt — let the maker complete at least three iterations independently

✔ Correct Answer: B) 'Never edit check.py to make it pass' — this is exactly what a loop optimizing for green will reach for. Noticing the urge is the lesson: the measure is the target, and teaching yourself to stop when the loop is about to game its own checker is the real skill

Explanation: 'One rule the project will not bend on, and it is the sharpest thing in it: never edit check.py to make it pass. That is exactly what a loop optimising for green will reach for, and noticing the urge is the lesson.'

Q79. The course says 'skill keeps loop prompts tiny' and gives a specific reason why detail should NOT be in the Routine's prompt. What is it?

A) Routines have a character limit on prompts that prevents detailed instructions

B) A long block of instructions in the schedule has nobody to keep it up to date — errors in it persist silently through every run. The skill holds the detail and is easy to update separately. Also: the prompt is read and paid for on every beat, so keeping it short directly reduces per-beat token cost

C) Anthropic's servers process short prompts faster, reducing latency on each Routine run

D) Long prompts in Routines trigger the AI's self-censorship mechanisms more frequently, reducing reliability

✔ Correct Answer: B) A long block of instructions in the schedule has nobody to keep it up to date — errors in it persist silently through every run. The skill holds the detail and is easy to update separately. Also: the prompt is read and paid for on every beat, so keeping it short directly reduces per-beat token cost

Explanation: 'Do not paste a long block of instructions into a schedule that nobody will keep up to date.' Two reasons: (1) update problem — nobody maintains it; (2) token cost — 'keep loop prompts and the rules file short: you pay for them on every beat.'

Q80. The course explains why 'the model is a second cost lever in OpenCode.' What is the practical pattern it recommends, and what is the important caveat?

A) Use the cheapest model for all tasks — cost savings always outweigh quality loss in loop contexts

B) Lower-cost maker, trustworthy checker — use a cheaper model for clear mechanical work, keep tests/linters/a reliable reviewer as the checker. Caveat: a weaker maker may produce more failed attempts, potentially removing the savings. AND frequency still matters most — a model 30x cheaper but running every 5 minutes can still cost more than Sonnet running hourly

C) Alternate between expensive and cheap models on alternate beats to balance cost and quality

D) Use the cheapest model for all tasks except the checker, which always uses the frontier model regardless of cost

✔ Correct Answer: B) Lower-cost maker, trustworthy checker — use a cheaper model for clear mechanical work, keep tests/linters/a reliable reviewer as the checker. Caveat: a weaker maker may produce more failed attempts, potentially removing the savings. AND frequency still matters most — a model 30x cheaper but running every 5 minutes can still cost more than Sonnet running hourly

Explanation: 'A practical pattern is lower-cost maker, trustworthy checker... However, a weaker maker may produce more failed attempts, which can remove the savings.' And: 'Frequency still matters most. A model that is 30 times cheaper but runs every five minutes can still cost more.'

Q81. The 'Self-check' question in the course asks: 'In the morning-triage loop, what stops a wrong fix from being merged while you sleep?' What is the complete answer?

A) The token cost limit prevents the loop from processing enough files to cause serious damage

B) Multiple layers: (1) the reviewer subagent must give PASS — if FAIL, no PR is opened; (2) the claude/ branch rule means the loop can only push to claude/ branches, never directly to main; (3) 'when in doubt, escalate' — the loop writes FAIL items to progress.md for human review; (4) the human gate — you review PRs before merging

C) The daily Routine cap prevents more than 5 PRs from being opened in a single day

D) The worktree isolation ensures each fix is in its own branch, making merges require explicit human action

✔ Correct Answer: B) Multiple layers: (1) the reviewer subagent must give PASS — if FAIL, no PR is opened; (2) the claude/ branch rule means the loop can only push to claude/ branches, never directly to main; (3) 'when in doubt, escalate' — the loop writes FAIL items to progress.md for human review; (4) the human gate — you review PRs before merging

Explanation: The answer combines: reviewer must PASS (else no PR), claude/ branch guardrail (can't push to main), escalation to progress.md for risky items, and the human gate at PR merge time. 'What stops a wrong fix from being merged while you sleep' = all four working together.

Q82. The course describes the 'LLM-as-judge' pattern. What is the key requirement for this to be trustworthy, and what makes a rubric checker weaker than a test checker?

A) LLM-as-judge requires the judge to use a larger model than the maker — quality scales with model size

B) The key requirement: the judge must not be the maker — 'a model that checks its own output often approves it too easily.' A rubric checker is weaker because a rubric score is a CLAIM (the model's judgment), not a PROOF (a test that passes or fails). 'A passing test earns autonomy. A rubric score does not'

C) LLM-as-judge requires showing the judge the full conversation history to detect inconsistencies across turns

D) LLM-as-judge is only trustworthy when the judge uses a different organization's model (e.g., GPT-4 judging Claude)

✔ Correct Answer: B) The key requirement: the judge must not be the maker — 'a model that checks its own output often approves it too easily.' A rubric checker is weaker because a rubric score is a CLAIM (the model's judgment), not a PROOF (a test that passes or fails). 'A passing test earns autonomy. A rubric score does not'

Explanation: 'A model that checks its own output often approves it too easily. A second agent uses different instructions and may use a different model.' And: 'a score from a model is still a claim, not a proof, and it is weaker than a passing test... The weaker your checker, the more work must pass through the human gate.'

Q83. Why does the course say the GitHub event hourly cap in Routines makes a 'nightly reconciliation sweep' necessary?

A) The sweep refreshes authentication tokens that expire after each hourly cap reset

B) GitHub events past the hourly cap are DROPPED, not queued — they are silently missed until the window resets. So an event-heavy design cannot rely solely on events and needs a nightly scheduled run that catches anything the events missed: 'dropped, not queued' is the critical detail

C) The sweep processes the batch of events that accumulated during the cap period once it resets

D) The nightly sweep reduces the total event volume to stay within the cap by pre-filtering irrelevant events

✔ Correct Answer: B) GitHub events past the hourly cap are DROPPED, not queued — they are silently missed until the window resets. So an event-heavy design cannot rely solely on events and needs a nightly scheduled run that catches anything the events missed: 'dropped, not queued' is the critical detail

Explanation: 'During the preview, GitHub events have per-routine and per-account hourly caps, and events past the cap are dropped until the window resets. Dropped, not queued. So an event-heavy design needs a reconciliation sweep: a nightly scheduled run that catches anything the events missed.'

Q84. The course describes a 'fail loudly at the limit' requirement for unattended loops. What specific outputs must the loop produce when it hits its cap or errors?

A) Send an automated support ticket to the development team's ticketing system

B) Leave a clear 'needs a human' note — not just stop silently. AND: 'Write a line every run, even on failure: each beat appends a note with a timestamp to progress.md covering what it tried, what passed, and what broke. A silent failure is the worst kind'

C) Automatically restart with a smaller scope to retry within a lower limit

D) Create a GitHub issue tagged 'automation-failure' with the error details

✔ Correct Answer: B) Leave a clear 'needs a human' note — not just stop silently. AND: 'Write a line every run, even on failure: each beat appends a note with a timestamp to progress.md covering what it tried, what passed, and what broke. A silent failure is the worst kind'

Explanation: 'Fail loudly at the limit: when the loop hits its cap or errors, it should leave a clear needs-a-human note, not just stop.' And: 'Write a line every run, even on failure... A silent failure is the worst kind.'

Q85. The course says 'a loop you cannot debug is a loop you cannot trust.' What three specific mechanisms for keeping loops debuggable does it name?

A) Comprehensive unit tests, integration tests, and end-to-end tests for the loop code

B) Send output where you will see it (log file, Slack/Discord via Channels, Triage inbox — not the terminal you already closed); write a line every run even on failure (timestamped entry to progress.md); keep runs replayable (opencode run --format json, opencode export, opencode session list in OpenCode; Routines run history and --resume in Claude Code)

C) Version control for all loop prompts, semantic versioning for skill files, and change log for connector configurations

D) Real-time monitoring dashboard, automated alerting thresholds, and on-call rotation for loop failures

✔ Correct Answer: B) Send output where you will see it (log file, Slack/Discord via Channels, Triage inbox — not the terminal you already closed); write a line every run even on failure (timestamped entry to progress.md); keep runs replayable (opencode run --format json, opencode export, opencode session list in OpenCode; Routines run history and --resume in Claude Code)

Explanation: The three debuggability mechanisms: (1) send output where you will see it, (2) write a line every run even on failure, (3) keep runs replayable. 'A loop you cannot debug is a loop you cannot trust.'

Q86. The course describes a progression for earning autonomy: 'report-only → fixes behind human gate → unattended action.' What does this progression track?

A) The progression from using free plans to paid plans as loop complexity increases

B) The trust ladder — a loop earns each level by being right at the level below. 'Prove the loop before overnight use: grow a loop on two axes at once. Cadence: run it hourly and watched for a few days before you let it run nightly and unattended. Capability: start report-only, then allow fixes behind the human gate, and only then unattended action'

C) The progression from junior engineer to senior engineer in terms of AI tool proficiency

D) The progression from prototype to production in terms of system reliability metrics

✔ Correct Answer: B) The trust ladder — a loop earns each level by being right at the level below. 'Prove the loop before overnight use: grow a loop on two axes at once. Cadence: run it hourly and watched for a few days before you let it run nightly and unattended. Capability: start report-only, then allow fixes behind the human gate, and only then unattended action'

Explanation: The two axes of proving a loop: cadence (hourly+watched → nightly+unattended) and capability (report-only → fixes behind gate → unattended action). 'A loop earns each level by being right at the level below.'

Q87. The course says /web-setup is a common source of failed first Routines. Why?

A) /web-setup creates a Routine with the wrong schedule format

B) /web-setup grants only clone access and does NOT install the Claude GitHub App on the repository. For GitHub-trigger Routines, the Claude GitHub App must be installed on the repo — and this specific difference has 'caused many failed first tries' where setup looks complete but the Routine cannot receive events

C) /web-setup creates environment variables that conflict with the Routine's connector settings

D) /web-setup requires admin permissions that most developers don't have, causing authentication failures

✔ Correct Answer: B) /web-setup grants only clone access and does NOT install the Claude GitHub App on the repository. For GitHub-trigger Routines, the Claude GitHub App must be installed on the repo — and this specific difference has 'caused many failed first tries' where setup looks complete but the Routine cannot receive events

Explanation: Appendix A3: 'Setup: The Claude GitHub App has to be installed on the repo. Careful: /web-setup only grants clone access, and it does not install the app. This causes many failed first tries.'

Q88. The course describes mcp_servers added locally with 'claude mcp add' as invisible to Routines. Why, and what are the two solutions?

A) Local MCP servers use different authentication than cloud Routines; re-authenticate them through the web interface

B) Locally-added MCP servers live on YOUR machine, not your account — they are invisible to cloud Routines that run on Anthropic's servers. Two solutions: (1) add them as connectors at claude.ai/customize/connectors, or (2) declare them in a committed .mcp.json file so they travel with the repository clone

C) Local MCP servers have insufficient permissions for cloud execution; grant elevated permissions through the admin panel

D) Local MCP servers have no web API, making them inaccessible to cloud Routines; replace with hosted MCP alternatives

✔ Correct Answer: B) Locally-added MCP servers live on YOUR machine, not your account — they are invisible to cloud Routines that run on Anthropic's servers. Two solutions: (1) add them as connectors at claude.ai/customize/connectors, or (2) declare them in a committed .mcp.json file so they travel with the repository clone

Explanation: Appendix A2: 'MCP servers you added locally with claude mcp add live on your machine, not your account, so they are invisible to a routine. Either add them as connectors at claude.ai/customize/connectors, or declare them in a committed .mcp.json so they travel with the clone.'

Q89. The course describes 'unrestricted branch pushes' as a setting that should be left OFF by default. What is the connection to the human gate concept?

A) Unrestricted pushes disable branch protection rules, removing GitHub's automated safeguards

B) The default claude/ branch prefix is a standing permission boundary AND a human gate: the loop cannot push to main without explicit human action (PR + merge). Enabling unrestricted branch pushes removes this gate for that repository — 'an unattended agent pushing to main on a bad run is exactly what the human gate exists to prevent'

C) Unrestricted pushes allow the Routine to create new repositories, which could exceed storage limits

D) The setting enables force-pushing, which could overwrite commit history permanently

✔ Correct Answer: B) The default claude/ branch prefix is a standing permission boundary AND a human gate: the loop cannot push to main without explicit human action (PR + merge). Enabling unrestricted branch pushes removes this gate for that repository — 'an unattended agent pushing to main on a bad run is exactly what the human gate exists to prevent'

Explanation: Appendix A2: 'Leave unrestricted pushes off unless you have a specific, reviewed reason to allow them. An unattended agent pushing to main on a bad run is exactly what the human gate exists to prevent.'

Q90. The course describes 'the Doorbell project's most common hidden failure': a green checkmark with no review posted. What causes this and how is it caught?

A) The GitHub Actions runner ran out of memory — increase the runner size in the workflow YAML

B) Missing one configuration setting causes the run to succeed and do the review internally but post nothing externally — green checkmark, no visible output. 'Miss one setting and the run succeeds, does the review, and posts nothing at all.' The project's README names the setting and the symptom — the only way to catch it is to verify the external output (the review comment), not the run status

C) GitHub rate limits prevent posting more than one review per minute — add a delay between runs

D) The Claude GitHub App posts reviews asynchronously and they appear 5-10 minutes after the green checkmark

✔ Correct Answer: B) Missing one configuration setting causes the run to succeed and do the review internally but post nothing externally — green checkmark, no visible output. 'Miss one setting and the run succeeds, does the review, and posts nothing at all.' The project's README names the setting and the symptom — the only way to catch it is to verify the external output (the review comment), not the run status

Explanation: 'One thing to expect, because it cost us an hour: a green checkmark does not mean it worked. Miss one setting and the run succeeds, does the review, and posts nothing at all. The project's README names the setting and the symptom.'

Q91. The course uses Sky Watch vs Paper Watch to illustrate when a loop needs a spine vs. when it does not. What is the deciding factor?

A) Sky Watch processes external API data (no spine needed); Paper Watch processes local files (spine needed)

B) Sky Watch reprints TODAY's asteroids on every run — it speaks even when nothing happened (an 'all clear' is the point of a watch). Paper Watch shows ONLY what is new since last run — without a spine tracking what it already showed, every paper returns as new every day. 'Both are daily Routines. But Sky Watch needs no memory, while Paper Watch cannot work without the spine. Same heartbeat, opposite memory need'

C) Sky Watch is event-driven (no spine needed); Paper Watch is scheduled (spine always needed for schedules)

D) Sky Watch uses an external API that maintains state; Paper Watch processes a stateless data stream

✔ Correct Answer: B) Sky Watch reprints TODAY's asteroids on every run — it speaks even when nothing happened (an 'all clear' is the point of a watch). Paper Watch shows ONLY what is new since last run — without a spine tracking what it already showed, every paper returns as new every day. 'Both are daily Routines. But Sky Watch needs no memory, while Paper Watch cannot work without the spine. Same heartbeat, opposite memory need'

Explanation: 'Sky Watch reprints today's asteroids every run and needs no memory, while Paper Watch shows only what is new and cannot work without the spine. Same heartbeat, opposite memory need. That contrast is exactly when a loop needs one.'

Q92. The course says 'Routines act as you.' What does this mean for actions involving external communications, and what real-world example does it give?

A) Routines impersonate your account for authentication purposes only; external communications are signed by Anthropic

B) Everything a Routine does — commits, PRs, Slack posts, Linear tickets, drafted emails — carries YOUR personal identity. 'A routine that replies to clients keeps replying as you while you are on vacation.' Think through the identity side of anything externally visible before scheduling it

C) Routines act using a shared Anthropic service account, not your personal account

D) Routines act as the GitHub App identity for code operations and your personal identity for communication operations

✔ Correct Answer: B) Everything a Routine does — commits, PRs, Slack posts, Linear tickets, drafted emails — carries YOUR personal identity. 'A routine that replies to clients keeps replying as you while you are on vacation.' Think through the identity side of anything externally visible before scheduling it

Explanation: Appendix A4: 'Routines belong to your individual account, count against your daily allowance, and everything they do — commits, PRs, Slack posts, Linear tickets, and drafted emails — carries your identity. A routine that replies to clients keeps replying as you while you are on vacation.'

Q93. The minimum safe loop checklist has seven items. The course says 'miss one and the loop is unsafe, forgetful, or invisible.' What does each failure mode map to?

A) Unsafe = missing limit; forgetful = missing state file; invisible = missing human gate

B) Unsafe = no isolated branch/worktree (parallel writes collide), no read-only checker (maker approves own work), no human gate (wrong action goes straight to main). Forgetful = no state file (loop repeats first step forever). Invisible = no log or notification (a failure overnight is silent). Both success condition AND limit prevent running forever (unsafe/expensive)

C) Unsafe = missing connectors; forgetful = missing skill; invisible = missing heartbeat

D) Unsafe = wrong model selection; forgetful = context rot; invisible = insufficient observability tooling

✔ Correct Answer: B) Unsafe = no isolated branch/worktree (parallel writes collide), no read-only checker (maker approves own work), no human gate (wrong action goes straight to main). Forgetful = no state file (loop repeats first step forever). Invisible = no log or notification (a failure overnight is silent). Both success condition AND limit prevent running forever (unsafe/expensive)

Explanation: The checklist maps each item to a specific failure: worktree (collision), read-only checker (self-approval), state file (amnesia), human gate (wrong merge), log/notification (silent failure). 'Miss one and the loop is unsafe, forgetful, or invisible.'

Q94. What does the ISS Watch project demonstrate about in-session loops that reading the concept alone cannot convey?

A) It demonstrates that AI can access real-time external APIs without any additional configuration

B) It makes two things viscerally real that text cannot: (1) /loop reads 'every minute' as the heartbeat — you wrote no schedule, no script, no URL; the project folder carries all of that so your prompt could be one sentence. (2) Close the terminal and the watching dies with it — that is not a bug, it is the definition. A heartbeat you have WATCHED fire is worth more than three you have READ about

C) It demonstrates the token cost difference between in-session and cloud-based loops

D) It shows how to configure MCP connectors for real-time data streams within Claude Code

✔ Correct Answer: B) It makes two things viscerally real that text cannot: (1) /loop reads 'every minute' as the heartbeat — you wrote no schedule, no script, no URL; the project folder carries all of that so your prompt could be one sentence. (2) Close the terminal and the watching dies with it — that is not a bug, it is the definition. A heartbeat you have WATCHED fire is worth more than three you have READ about

Explanation: 'Two things are worth noticing, because together they are the whole concept: /loop read every minute as the heartbeat... Now close the terminal. The watching dies with it.' The course says explicitly: 'A heartbeat you have watched fire is worth more than three you have read about.'

Q95. The course describes what happens when the same check moves from 'your personal infrastructure' to 'team infrastructure.' What is the specific mechanism and what does it prevent?

A) The check is merged into the main branch and applied as a mandatory code style rule

B) When the chain is solid for your own changes, the same skills run on every pull request through the event heartbeat (the Doorbell). 'A teammate's change now passes the same gates yours did, whether they remembered to invoke anything or not.' This prevents the gap where your standards apply to your work but not to the team's — the check becomes automatic for everyone

C) The check is published to the Claude Skills directory where all team members can install it voluntarily

D) The check is added to the CI/CD pipeline as a required status check that blocks merging on failure

✔ Correct Answer: B) When the chain is solid for your own changes, the same skills run on every pull request through the event heartbeat (the Doorbell). 'A teammate's change now passes the same gates yours did, whether they remembered to invoke anything or not.' This prevents the gap where your standards apply to your work but not to the team's — the check becomes automatic for everyone

Explanation: 'Once the chain is solid for your own changes, the same skills run on every pull request through the event heartbeat you built in Concept 7, meaning the Doorbell, with your check as its prompt. A teammate's change now passes the same gates yours did, whether they remembered to invoke anything or not.'

Q96. The course says the Doorbell project proves something about state and the spine using 'second-push behavior.' What does this prove?

A) It proves that the loop's second run is always faster than the first due to cached context

B) Push a second time and the new review correctly cites your earlier commits by hash, despite running on a machine that has never existed before and remembers nothing — it did NOT remember; it READ the repo. This proves the spine principle: 'memory' for an event-driven loop is not session state but the repository itself — the commits, the files, the PR history that persist across ephemeral runners

C) It proves that the Claude GitHub App maintains session continuity across multiple pushes to the same PR

D) It proves that GitHub's runner infrastructure reuses virtual machines for consecutive runs on the same repository

✔ Correct Answer: B) Push a second time and the new review correctly cites your earlier commits by hash, despite running on a machine that has never existed before and remembers nothing — it did NOT remember; it READ the repo. This proves the spine principle: 'memory' for an event-driven loop is not session state but the repository itself — the commits, the files, the PR history that persist across ephemeral runners

Explanation: 'Push a second time and the new review will cite your earlier commits by hash, correctly, despite running on a machine that has never existed before and remembers nothing. It did not remember. It read the repo. That is the spine, and you will meet it properly in Part 4.'

Q97. The skill-creator plugin is mentioned as the fastest way to write a verification skill. What does the course say is the right second step after using it?

A) Submit the skill to the Claude Skills directory for peer review before using it in production

B) Interview yourself — '/skill-creator Create a skill for verifying frontend changes end-to-end. Interview me about my workflow.' It asks, you answer, and the file appears. But: YOUR version differs from the standard best-practice version on a few specific points — those differences ARE the valuable part. The model already knows the standard; the project-specific knowledge is what you contribute

C) Run the skill against a test suite of 10 diverse inputs to validate its accuracy before deploying

D) Share the skill with a senior engineer for code review to ensure the verification logic is sound

✔ Correct Answer: B) Interview yourself — '/skill-creator Create a skill for verifying frontend changes end-to-end. Interview me about my workflow.' It asks, you answer, and the file appears. But: YOUR version differs from the standard best-practice version on a few specific points — those differences ARE the valuable part. The model already knows the standard; the project-specific knowledge is what you contribute

Explanation: 'The fastest way to write a verification skill is the skill-creator plugin... It asks, you answer, and the file appears. Hand-writing into .claude/skills/ works exactly as in Concept 9.' The key: your project's specific deviations from standard practice are the valuable content.

Q98. What is the relationship between the Append A6 'Routine version of the checklist' and the minimum safe loop checklist from Part 5?

A) The Routine checklist adds 10 new items specific to cloud deployment; the Part 5 checklist only applies to local loops

B) The seven items translate directly: success condition and limit live in the prompt; isolation is the claude/ branch prefix; read-only checker is a reviewer subagent in the cloned repo; state file is a committed context file (because the clone is fresh every time); human gate is the two-routine pattern or 'draft PRs only, never merge'; log is the run transcript plus a connector post. 'Green does not mean done' applies here too

C) The Routine checklist replaces the Part 5 checklist — cloud Routines have different safety requirements

D) The Routine checklist is a subset of the Part 5 checklist — only 4 of the 7 items apply to cloud Routines

✔ Correct Answer: B) The seven items translate directly: success condition and limit live in the prompt; isolation is the claude/ branch prefix; read-only checker is a reviewer subagent in the cloned repo; state file is a committed context file (because the clone is fresh every time); human gate is the two-routine pattern or 'draft PRs only, never merge'; log is the run transcript plus a connector post. 'Green does not mean done' applies here too

Explanation: Appendix A6: 'The minimum safe loop checklist translates directly' — same seven items, different implementation for Routines. Each item maps to a specific Routine configuration choice. The same principles, applied to the cloud-hosted context.

Q99. The course says 'the loop that improves the loop' is bigger than it looks. What does it mean practically when a loop WRITES A LESSON into CLAUDE.md?

A) The lesson is temporarily stored and will be discarded when the rules file is next consolidated

B) When the loop writes a lesson into CLAUDE.md so every future run behaves better, it is doing hill-climbing by hand: 'the output is not work, its output is improvements to the system that does the work.' This lesson persists because it cannot survive in the model but CAN survive on disk. 'The memory builds, the system gets sharper, and the gains add up, run after run'

C) The lesson triggers a model fine-tuning job that updates the agent's weights for future sessions

D) The lesson is submitted to Anthropic for potential inclusion in the next Claude model training dataset

✔ Correct Answer: B) When the loop writes a lesson into CLAUDE.md so every future run behaves better, it is doing hill-climbing by hand: 'the output is not work, its output is improvements to the system that does the work.' This lesson persists because it cannot survive in the model but CAN survive on disk. 'The memory builds, the system gets sharper, and the gains add up, run after run'

Explanation: 'When your loop writes a lesson into CLAUDE.md so every future run behaves better, you are doing something by hand that has a name: a hill-climbing loop. Its output is not work. Its output is improvements to the system that does the work.' The spine makes this possible — memory on disk, not in weights.

Q100. The course says Peter Steinberger's original quote was 'you should be designing loops that prompt your agents.' How does the course use this as its opening argument, and what does Addy Osmani's contribution add?

A) Steinberger described the technical implementation; Osmani added the business case for why loops are economically superior to prompting

B) Steinberger (OpenClaw) and Cherny (Claude Code) independently said the important skill moved — from the prompt to the loop. Osmani then gave the pattern a name and listed its parts. Together: the idea is not coming from one tool's creators, it is converging from multiple practitioners, which is why the course says 'both views are partly right' about whether it is new — 'the parts are not new, but they have become affordable and reliable enough for everyday use'

C) Steinberger focused on event-driven loops; Osmani added scheduled loops; Cherny added conditional loops — together they describe all four heartbeats

D) All three independently arrived at the same six-part structure, proving the structure is theoretically optimal

✔ Correct Answer: B) Steinberger (OpenClaw) and Cherny (Claude Code) independently said the important skill moved — from the prompt to the loop. Osmani then gave the pattern a name and listed its parts. Together: the idea is not coming from one tool's creators, it is converging from multiple practitioners, which is why the course says 'both views are partly right' about whether it is new — 'the parts are not new, but they have become affordable and reliable enough for everyday use'

Explanation: 'In 2026, the people who build these tools said it clearly... None of them say the work got easier. They say the important skill moved... Both views are partly right. The parts are not new, but they have become affordable and reliable enough for everyday use. As a result, the main job is increasingly to design the loop rather than guide every agent turn.'

CodeVerse Soban

End of MCQ Practice — Best of Luck!